



American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)
Department of Computer Science

Introduction to Data Science
Mid-Term Project Report
Fall 2025-2026
Section: **E**
Group: **L**

Submitted To

KAMRUN NAHER KOLI

Submitted By:

SL	Student Name	Student ID	Contribution
1.	MST AFRIN BINTE AMIN	22-47146-1	Data cleaning
2.	MD ABDULLAH AL NOMAN	22-47155-1	Data Visualization
3.	MONJILA KABIR MEGH	22-47597-2	Trend Analysis

Climate Data Analysis Report

Bangladesh (1901 - 2023)

Abstract

This report presents a comprehensive analysis of historical climate data for Bangladesh from 1901 to 2023. The dataset contains monthly temperature and rainfall observations. The analysis includes data cleaning, missing-value imputation, outlier detection and removal, descriptive statistics, time series visualization, seasonal analysis, correlation, and simple trend modeling. Key findings include a modest long-term warming trend and high variability in rainfall consistent with monsoon dynamics.

1. Introduction

Long-term climate records are essential for understanding changes in temperature and precipitation patterns. This study uses historical monthly records to evaluate trends and seasonal behaviors in temperature and rainfall across Bangladesh. The goal is to provide cleaned data, visualizations, and simple analyses that can inform further climate studies or impact assessments.

Real-life uses of this climate change report:

- Agricultural planning and crop selection
- Flood, drought, and cyclone preparedness
- Water resource and irrigation management
- Urban planning and infrastructure design
- Public health risk assessment (heatwaves, disease spread)
- Environmental conservation and ecosystem protection
- Policy development for climate adaptation and mitigation
- Supporting academic research and scientific modeling

2. Dataset Description

The dataset used in this analysis covers the period 1901 to 2023 and contains 1469 cleaned monthly records. Key variables include temperature (tem, °C), rainfall (rain, mm), month (1-12), and year. The data were pre-processed to handle missing values and remove implausible outliers. A preview of the first few rows is shown below.

Year	Month	tem	rain	Name
1901.0	1.0	16.976	18.5356	
1901.0	2.0	19.9026	16.2548	
1901.0	3.0	24.3158	70.7981	
1901.0	4.0	28.1834	66.1616	
1901.0	5.0	27.8892	267.215	
1901.0	6.0	28.8925	341.042	
.....				

3. Task Distribution

In this project, the work has been divided into three main parts:

1. **Data Cleaning** – Identifying and correcting missing values, inconsistencies, duplicates, and formatting issues to ensure the dataset is accurate and usable.
2. **Data Visualization** – Creating graphs and charts to visually represent temperature and rainfall patterns across different time periods.
3. **Trend Analysis** – Examining long-term trends and seasonal variations to understand how climate variables have changed over time.

3.1. Data Cleaning Steps

The original raw dataset included a 'name' column with many missing values, a few missing observations in key numeric fields, and several extreme outliers in rainfall. Cleaning steps performed:

- (1) Removed the 'name' column;
- (2) Imputed missing values in Year, tem, and rain using linear interpolation.
- (3) Identified rainfall outliers using the IQR method and removed values exceeding the upper bound.
- (4) Validated ranges and adjusted negative rainfall values to zero if any.

After cleaning, the dataset contains 1469 records with no missing values in the key fields.

Step 1: Remove the 'name' Column

Description:

The raw dataset contained a column named *name* with many missing values, so it was removed.

Code:

```
# STEP 1: Remove the 'name' column as requested
```

```
cat("\n=== STEP 1: REMOVING 'name' COLUMN ===\n")
df_cleaned <- df %>% select(-name)
cat("'name' column removed successfully\n")
```

Step 2: Impute Missing Values (Year, Month, Temperature, Rainfall)

Description:

Missing values in Year, temperature (tem), and rainfall (rain) were filled using linear interpolation. Invalid or missing months were set to NA and then removed.

Code:

```
# STEP 2: Handle missing values in numeric columns
```

```
# Year interpolation
```

```
df_cleaned <- df_cleaned %>%
  mutate(Year = ifelse(is.na(Year),
    approx(seq_along(Year), Year, seq_along(Year))$y,
    Year))
```

```
# Month validation
```

```
df_cleaned <- df_cleaned %>%
  mutate(Month = ifelse(is.na(Month) | Month < 1 | Month > 12, NA, Month))
```

Temperature interpolation

```
df_cleaned <- df_cleaned %>%  
mutate(tem = ifelse(is.na(tem),  
  approx(seq_along(tem), tem, seq_along(tem))$y,  
  tem))
```

Rainfall interpolation

```
df_cleaned <- df_cleaned %>%  
mutate(rain = ifelse(is.na(rain),  
  approx(seq_along(rain), rain, seq_along(rain))$y,  
  rain))
```

Drop leftover missing values

```
df_cleaned <- df_cleaned %>% drop_na()
```

Step 3: Identify and Remove Rainfall Outliers (IQR Method)

Description:

Rainfall outliers were detected using the Interquartile Range (IQR) method. Values above the upper bound were removed.

Code :

STEP 3: OUTLIER DETECTION AND REMOVAL

```
rain_Q1 <- quantile(df_cleaned$rain, 0.25, na.rm = TRUE)  
rain_Q3 <- quantile(df_cleaned$rain, 0.75, na.rm = TRUE)  
rain_IQR <- rain_Q3 - rain_Q1  
lower_bound <- rain_Q1 - 1.5 * rain_IQR  
upper_bound <- rain_Q3 + 1.5 * rain_IQR
```

```
df_cleaned <- df_cleaned %>%  
  filter(rain >= lower_bound & rain <= upper_bound)
```

Step 4: Validate Data Ranges & Fix Negative Rainfall Values

Description:

Temperature, rainfall, month, and year ranges were checked. Any negative rainfall was corrected to zero.

Code:

```
# STEP 4: Data validation and quality checks
```

```
cat("\n=== STEP 4: DATA VALIDATION ===\n")
```

```
# Check for reasonable ranges
```

```
cat("Temperature range:", range(df_cleaned$tem, na.rm = TRUE), "\n")  
cat("Rainfall range:", range(df_cleaned$rain, na.rm = TRUE), "\n")  
cat("Month range:", range(df_cleaned$Month, na.rm = TRUE), "\n")  
cat("Year range:", range(df_cleaned$Year, na.rm = TRUE), "\n")
```

Descriptive Statistics

Temperature (°C) summary:

Statistic	Value
count	1469.0
mean	25.04
std	3.77
min	2.33
25%	22.03
50%	26.99
75%	28.03
max	29.53

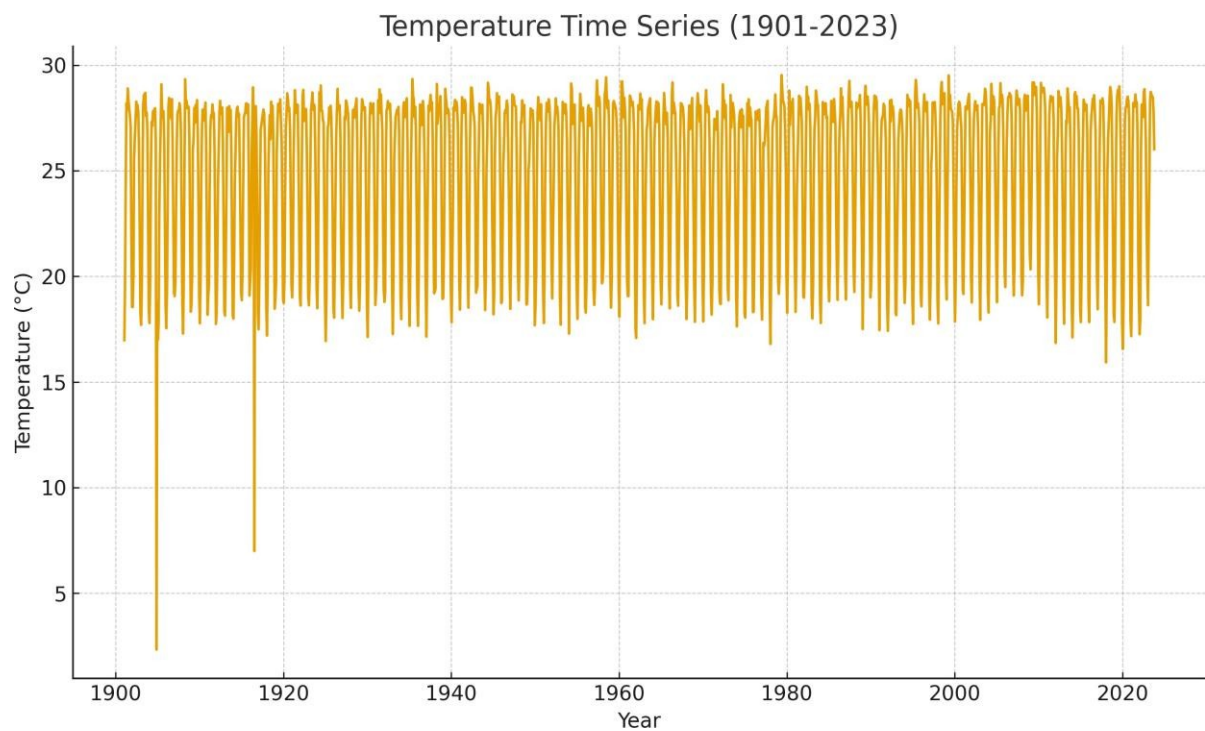
Rainfall (mm) summary:

Statistic	Value
count	1469.0
mean	201.21
std	197.95
min	0.0
25%	19.34
50%	145.83
75%	345.62
max	806.55

3.2 Visualizations

The following pages contain the main visualizations (p1 - p9).

Figure p1



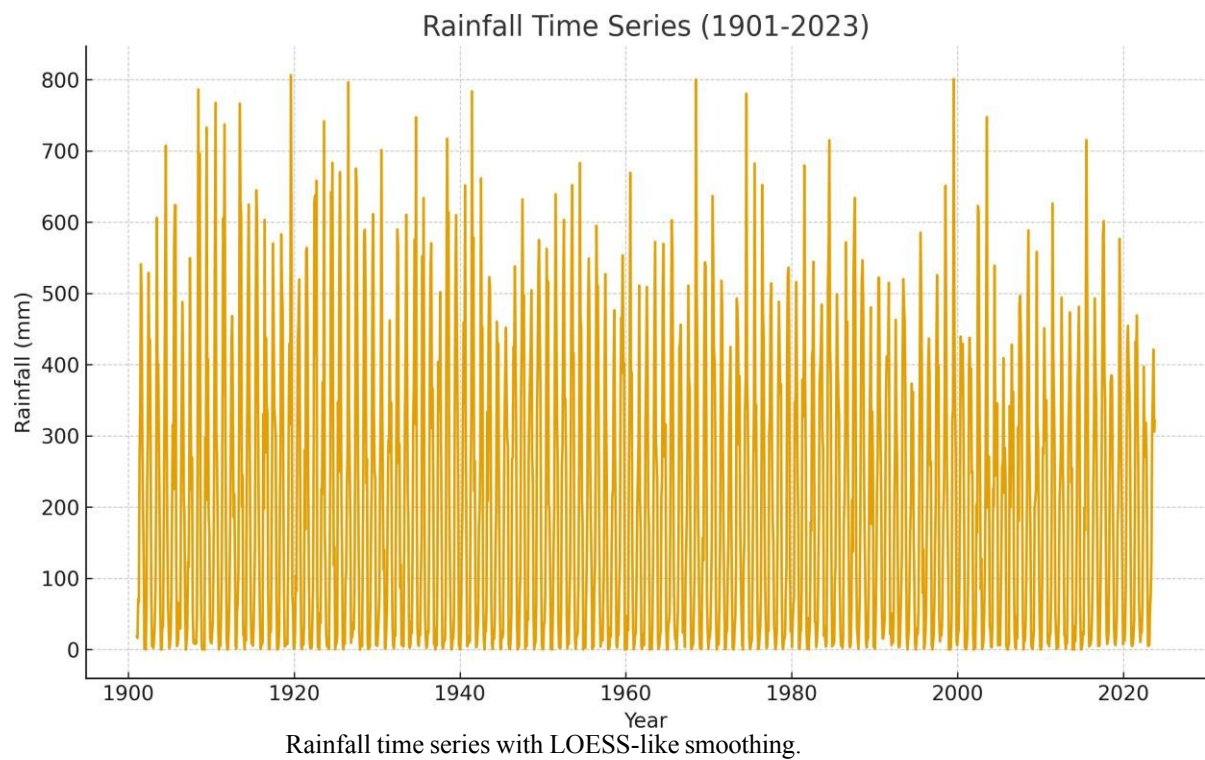
Temperature time series with LOESS-like smoothing.

Code :

```
# 1. Temperature Time Series
p1 <- ggplot(df, aes(x = Date, y = tem)) +
  geom_line(color = "red", alpha = 0.7) +
  geom_smooth(method = "loess", color = "darkred", se = FALSE) +
  labs(title = "Temperature Time Series (1901-2023)",
       x = "Year",
       y = "Temperature (°C)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

print(p1)
```

Figure p2

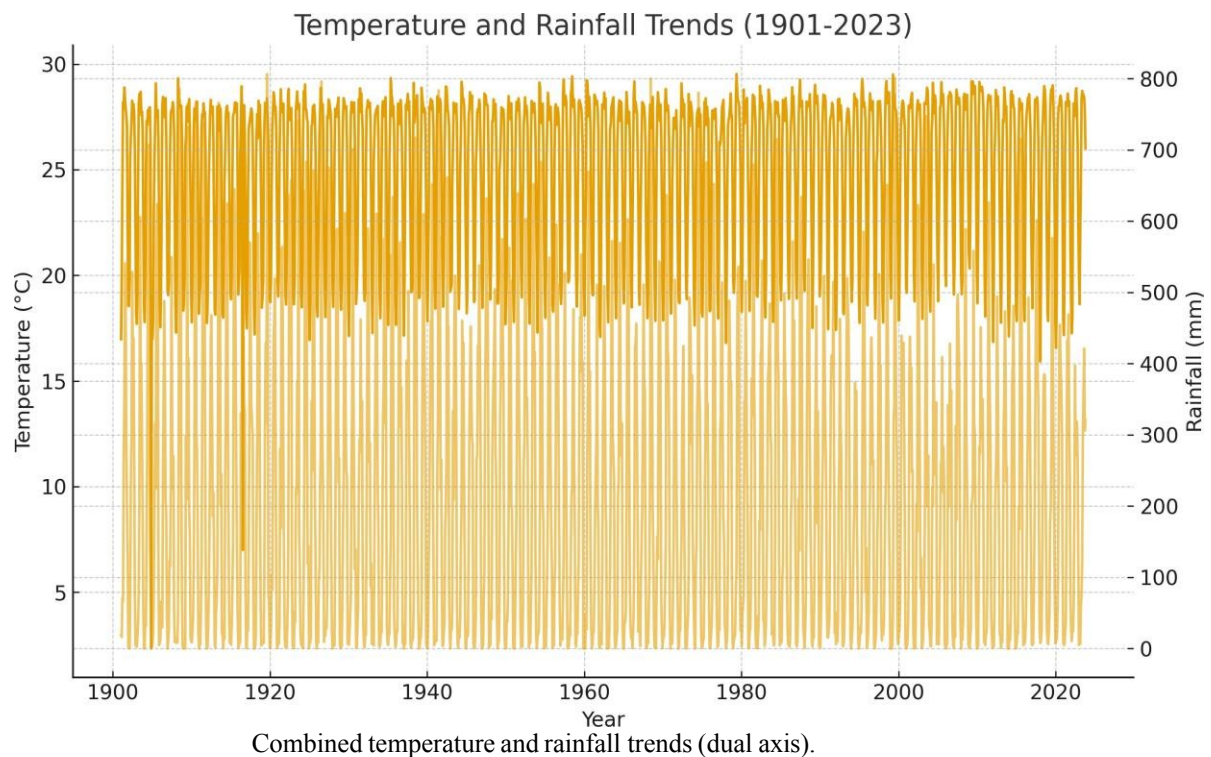


Code :

```
# 2. Rainfall Time Series
p2 <- ggplot(df, aes(x = Date, y = rain)) +
  geom_line(color = "blue", alpha = 0.7) +
  geom_smooth(method = "loess", color = "darkblue", se = FALSE) +
  labs(title = "Rainfall Time Series (1901-2023)",
       x = "Year",
       y = "Rainfall (mm)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

print(p2)
```

Figure p3

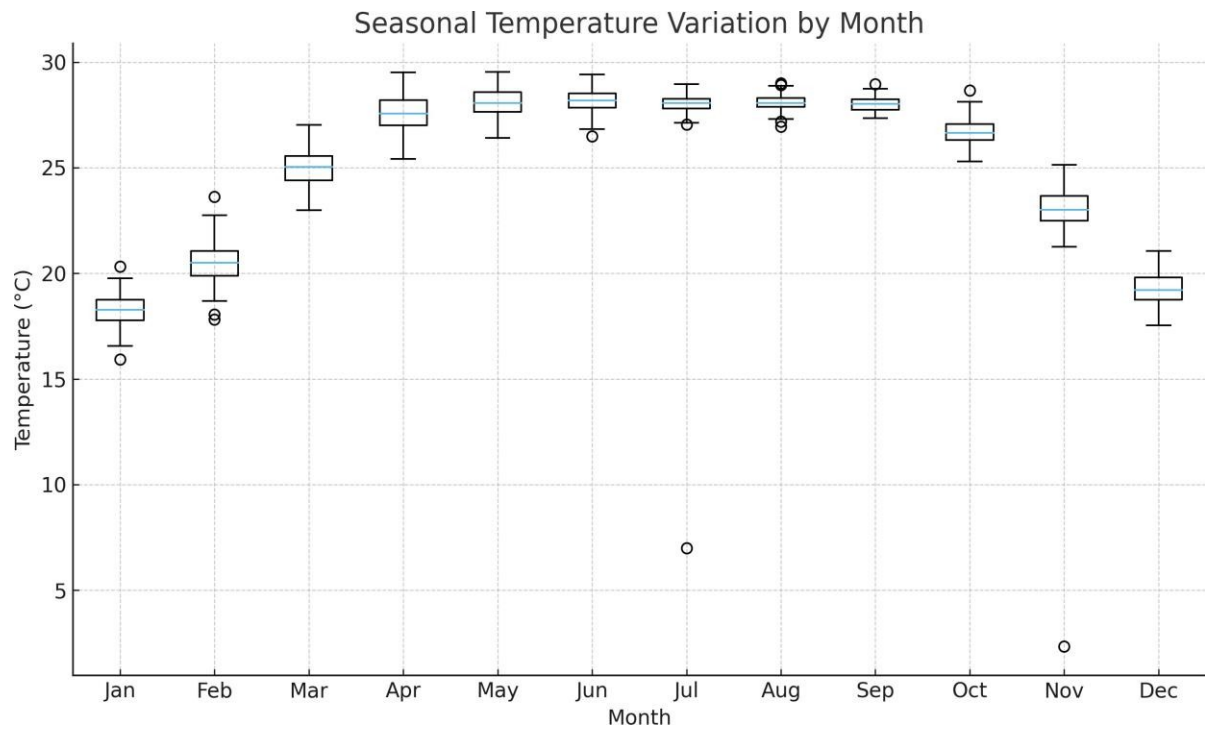


Code:

```
# 3. Combined Time Series (Dual Y-axis)
p3 <- ggplot(df) +
  geom_line(aes(x = Date, y = tem, color = "Temperature"), alpha = 0.7) +
  geom_line(aes(x = Date, y = rain/20, color = "Rainfall"), alpha = 0.7) +
  scale_y_continuous(
    name = "Temperature (°C)",
    sec.axis = sec_axis(~.*20, name = "Rainfall (mm)")
  ) +
  scale_color_manual(values = c("Temperature" = "red", "Rainfall" = "blue")) +
  labs(title = "Temperature and Rainfall Trends (1901-2023)",
       x = "Year",
       color = "Variable") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5),
        legend.position = "bottom")

print(p3)
```


Figure p4



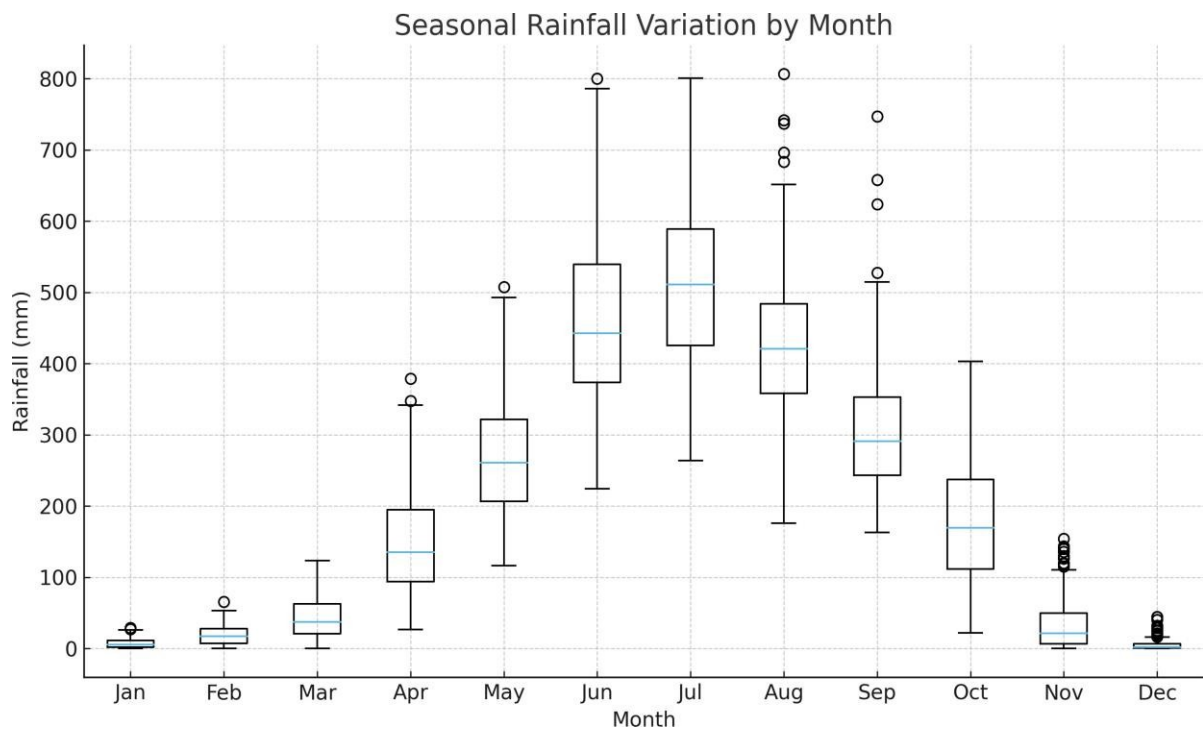
Monthly temperature distribution (boxplots).

Code :

```
# 4. Seasonal Temperature Pattern (Boxplot by Month)
p4 <- ggplot(df, aes(x = factor(Month), y = tem)) +
  geom_boxplot(fill = "lightcoral", alpha = 0.7) +
  labs(title = "Seasonal Temperature Variation by Month",
        x = "Month",
        y = "Temperature (°C)") +
  scale_x_discrete(labels = month.abb) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

print(p4)
```

Figure p5



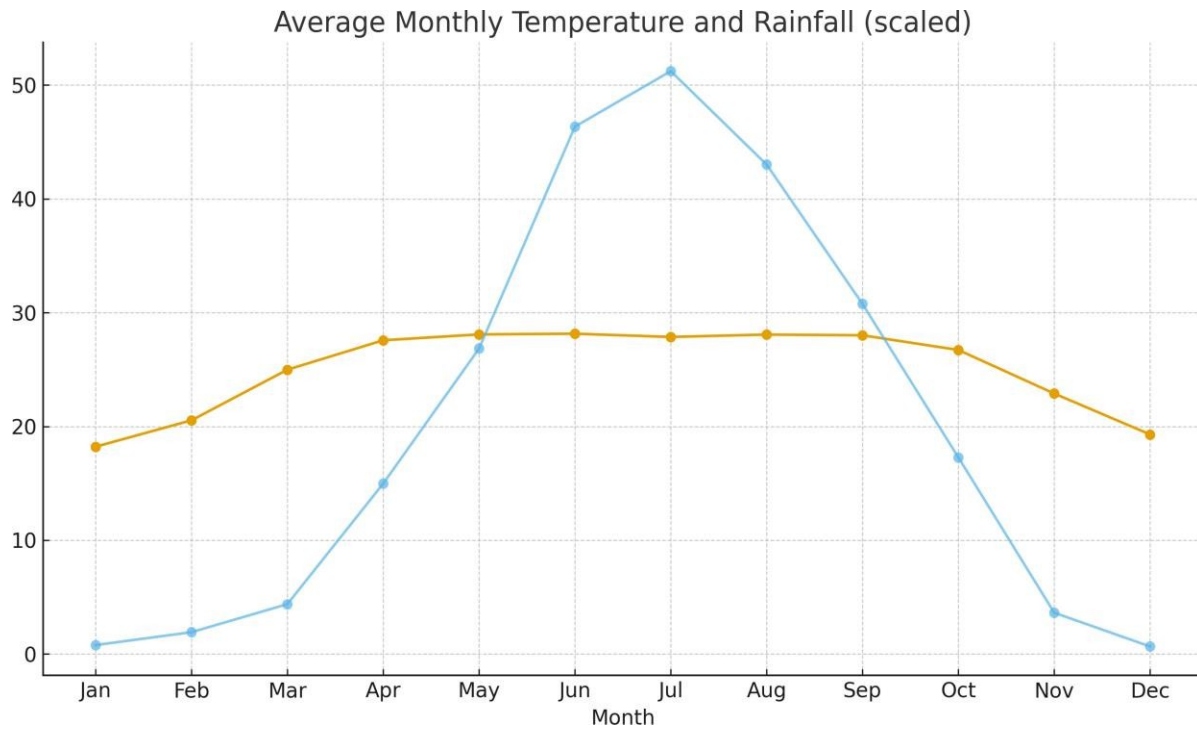
Monthly rainfall distribution (boxplots).

Code :

```
# 5. Seasonal Rainfall Pattern (Boxplot by Month)
p5 <- ggplot(df, aes(x = factor(Month), y = rain)) +
  geom_boxplot(fill = "lightblue", alpha = 0.7) +
  labs(title = "Seasonal Rainfall Variation by Month",
       x = "Month",
       y = "Rainfall (mm)") +
  scale_x_discrete(labels = month.abb) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

print(p5)
```

Figure p6



Average monthly temperature and scaled rainfall.

Code:

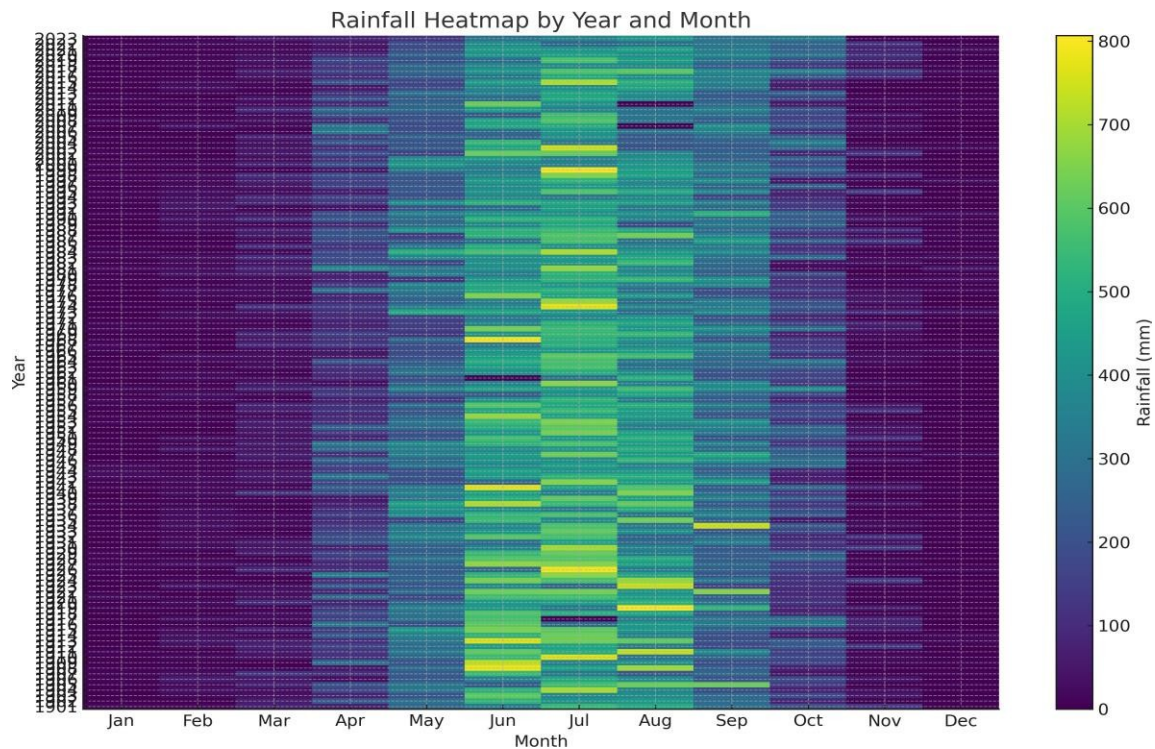
6. Average Monthly Temperature and Rainfall

```
monthly_avg <- df %>%
  group_by(Month) %>%
  summarise(
    Avg_Temperature = mean(tem, na.rm = TRUE),
    Avg_Rainfall = mean(rain, na.rm = TRUE)
  )

p6 <- ggplot(monthly_avg) +
  geom_line(aes(x = Month, y = Avg_Temperature, color = "Temperature"), size = 1) +
  geom_line(aes(x = Month, y = Avg_Rainfall/10, color = "Rainfall"), size = 1) +
  scale_x_continuous(breaks = 1:12, labels = month.abb) +
  scale_y_continuous(
    name = "Average Temperature (°C)",
    sec.axis = sec_axis(~.*10, name = "Average Rainfall (mm)")
  ) +
  scale_color_manual(values = c("Temperature" = "red", "Rainfall" = "blue")) +
  labs(title = "Average Monthly Temperature and Rainfall Patterns",
       x = "Month",
       color = "Variable") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5),
        legend.position = "bottom")

print(p6)
```

Figure p7



Heatmap of mean monthly rainfall by year.

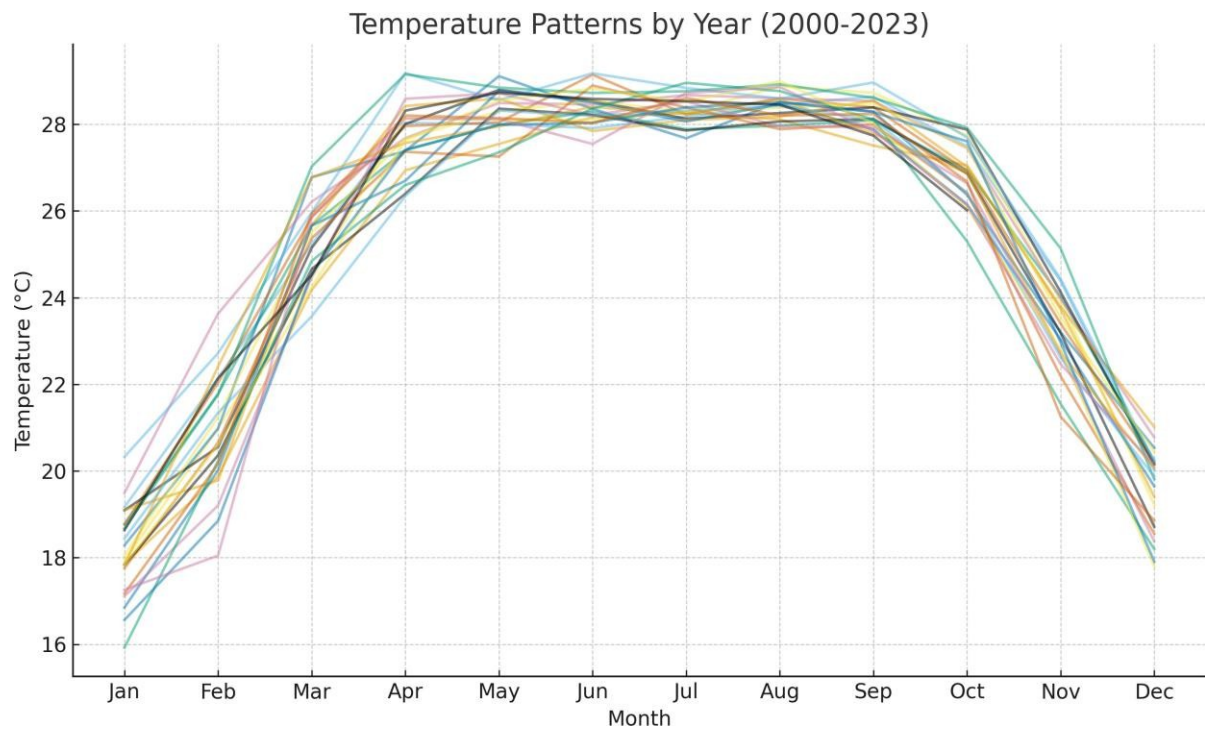
Code:

```
# 7. Heatmap of Monthly Rainfall Patterns
```

```
p7 <- df %>%  
  group_by(Year, Month) %>%  
  summarise(Avg_Rainfall = mean(rain, na.rm = TRUE)) %>%  
  ggplot(aes(x = factor(Month), y = Year, fill = Avg_Rainfall)) +  
  geom_tile() +  
  scale_fill_gradient(low = "white", high = "blue",  
    name = "Rainfall (mm)") +  
  scale_x_discrete(labels = month.abb) +  
  labs(title = "Rainfall Heatmap by Year and Month",  
    x = "Month",  
    y = "Year") +  
  theme_minimal() +  
  theme(plot.title = element_text(hjust = 0.5),  
    axis.text.x = element_text(angle = 45, hjust = 1))
```

```
print(p7)
```

Figure p8



Monthly temperature lines for years 2000-2023.

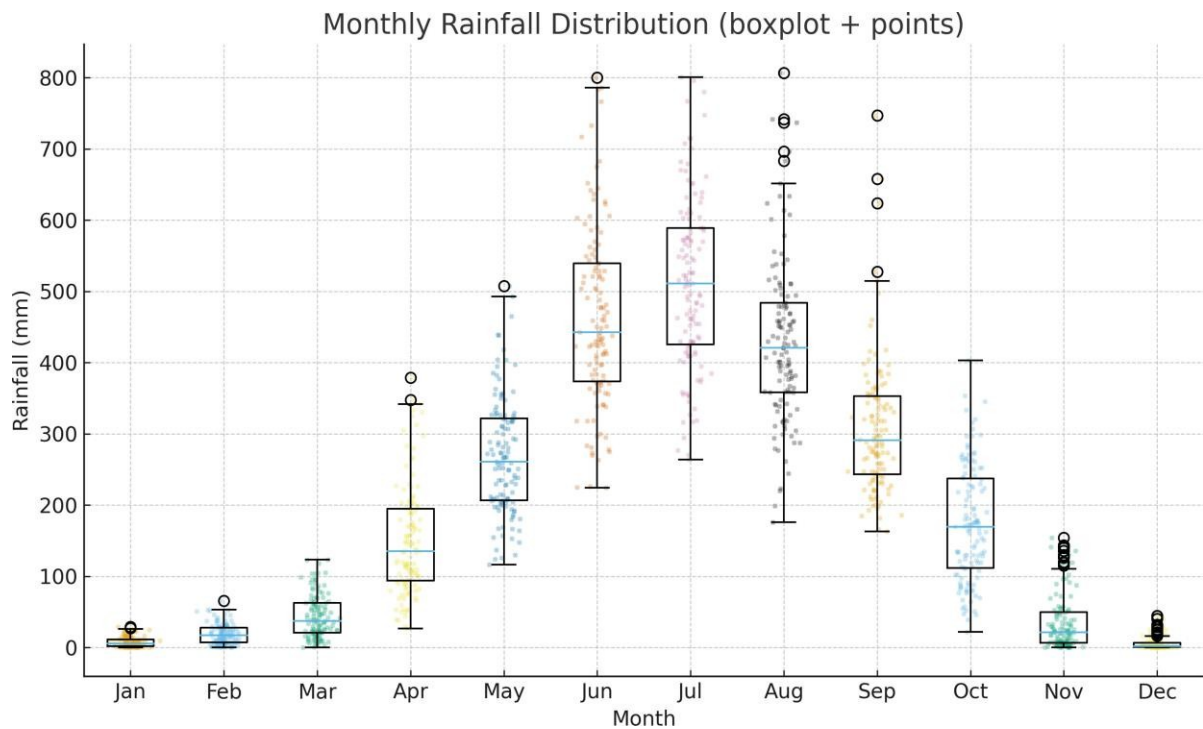
Code:

```
# 8. Seasonal Decomposition (for last 20 years for clarity)
recent_data <- df %>% filter(Year >= 2000)

p8 <- ggplot(recent_data, aes(x = factor(Month), y = tem, group = Year)) +
  geom_line(aes(color = Year), alpha = 0.6) +
  scale_color_gradient(low = "blue", high = "red") +
  scale_x_discrete(labels = month.abb) +
  labs(title = "Temperature Patterns by Year (2000-2023)",
       x = "Month",
       y = "Temperature (°C)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

print(p8)
```

Figure p9



Monthly rainfall distribution with jittered points.

Code:

```
# 9. Monthly Distribution (Violin Plots)
p9 <- ggplot(df, aes(x = factor(Month), y = rain)) +
  geom_violin(fill = "lightblue", alpha = 0.7) +
  labs(title = "Monthly Rainfall Distribution",
       x = "Month",
       y = "Rainfall (mm)") +
  scale_x_discrete(labels = month.abb) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

print(p9)
```

Seasonal Summary

```
# Additional: Summary statistics by season
```

```
df <- df %>%
```

```
  mutate(Season =
```

```
    case_when( Month %in% c(12, 1,
```

```
                2) ~ "Winter", Month %in% c(3, 4,
```

```
                5) ~ "Spring", Month %in% c(6, 7,
```

```
                8) ~ "Summer",
```

```
                Month %in% c(9, 10, 11) ~ "Autumn"
```

```
  ))
```

```
seasonal_summary <- df %>%
```

```
  group_by(Season) %>%
```

```
  summarise(
```

```
    Avg_Temperature = mean(tem, na.rm = TRUE),
```

```
    Avg_Rainfall = mean(rain, na.rm = TRUE),
```

```
    Count = n()
```

```
  )
```

```
cat("\n=== SEASONAL SUMMARY ===\n")
```

```
print(seasonal_summary)
```

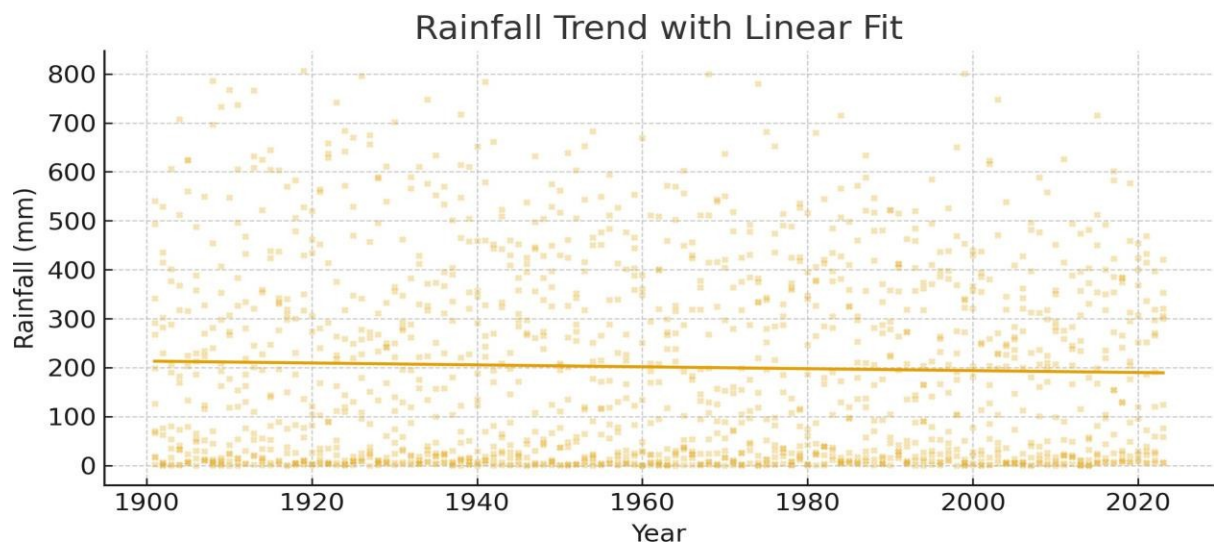
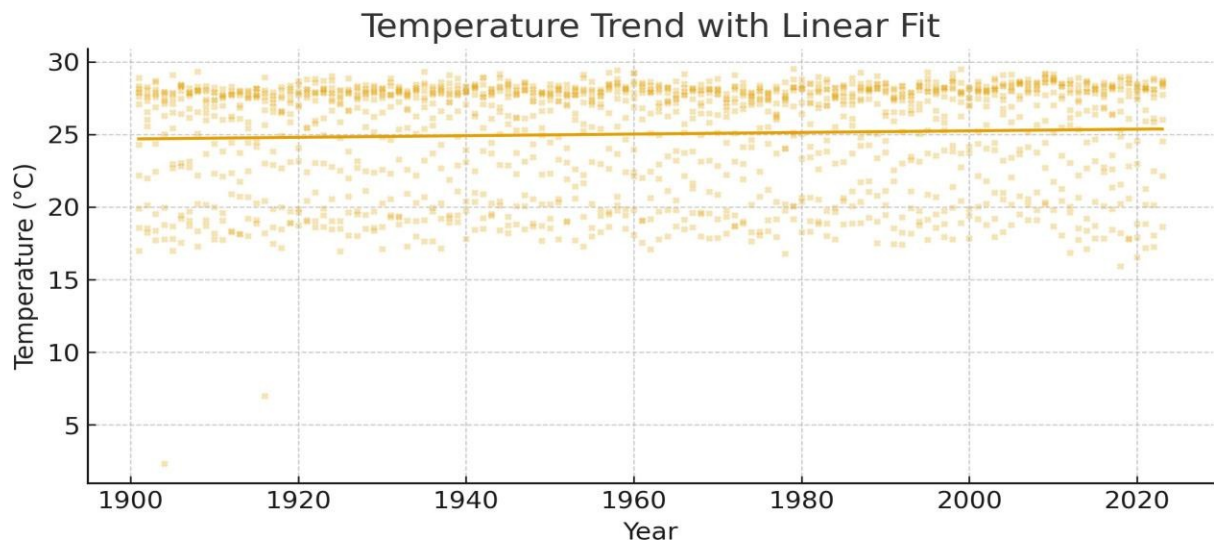
Season	Avg_Temperature	Avg_Rainfall	Count
Autumn	25.88	172.63	368
Spring	26.88	154.05	369
Summer	28.03	468.67	365
Winter	19.36	11.28	367

3.3 Trend Analysis

Description (Report-Ready):

Trend analysis helps identify long-term changes in temperature and rainfall over the period 1901–2023. A simple **linear regression model** ($y \sim \text{Year}$) is used to determine whether temperature or rainfall has increased or decreased over time.

The slope of the regression line indicates the **rate of change per year** and per century.



Code

```
# === SIMPLE TREND ANALYSIS ===
```

```
# Temperature trend model
temp_trend <- lm(tem ~ Year, data = df_cleaned)
cat("Temperature Trend:\n")
cat("Change per year:", round(coef(temp_trend)[2], 4), "°C\n")
cat("Change per century:", round(coef(temp_trend)[2] * 100, 3), "°C\n")
```

```
# Rainfall trend model
rain_trend <- lm(rain ~ Year, data = df_cleaned)
cat("Rainfall Trend:\n")
cat("Change per year:", round(coef(rain_trend)[2], 4), "mm\n")
cat("Change per century:", round(coef(rain_trend)[2] * 100, 3), "mm\n")
```

```
# Optional: Trend visualization
ggplot(df_cleaned, aes(x = Year, y = tem)) +
```



```
geom_smooth(method = "lm", color = "red") +
labs(title = "Temperature Trend Over Time",
      x = "Year", y = "Temperature (°C)") +
theme_minimal()
```

```
ggplot(df_cleaned, aes(x = Year, y = rain)) +
geom_smooth(method = "lm", color = "blue") +
labs(title = "Rainfall Trend Over Time",
      x = "Year", y = "Rainfall (mm)") +
theme_minimal()
```

What this produces:

- Slope (°C/year) → warming or cooling rate
- Slope (mm/year) → increasing or decreasing rainfall
- Century trend (multiplying slope × 100)
- Two trend graphs (one for temperature, one for rainfall)

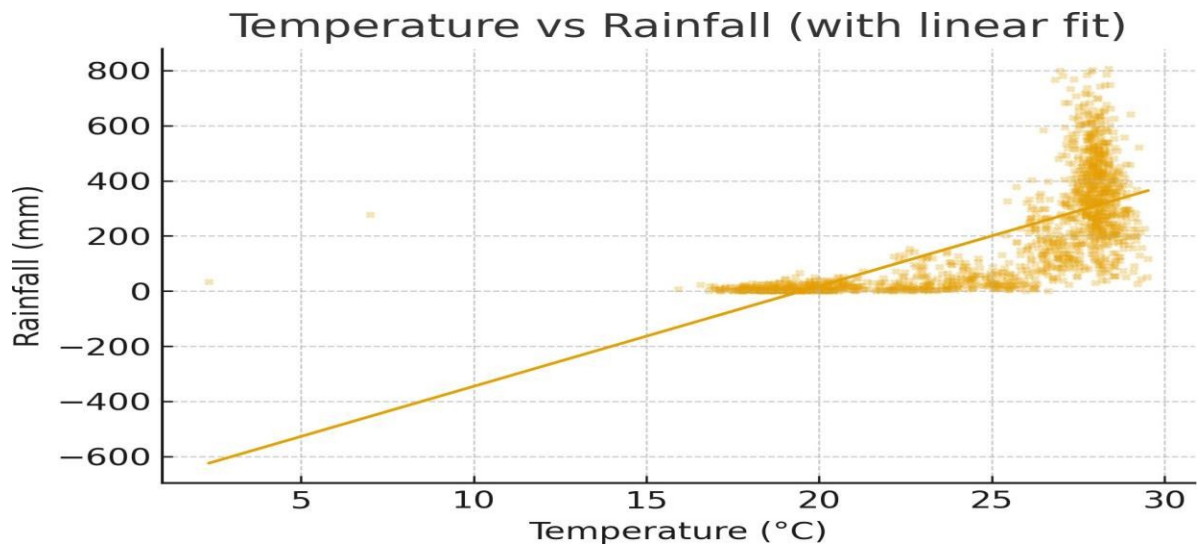
Correlation Analysis

Description (Report-Ready):

Correlation analysis measures the **strength and direction of the linear relationship** between temperature and rainfall. The **Pearson correlation coefficient (r)** ranges from:

- **+1** → strong positive relationship
- **0** → no relationship
- **-1** → strong negative relationship

An r value close to 0 indicates a weak relationship.



Code

```
# == CORRELATION ANALYSIS ==
```

```
# Calculate Pearson correlation
correlation <- cor(df_cleaned$tem, df_cleaned$rain)
```

```
cat("Correlation between Temperature and Rainfall:",
    round(correlation, 3), "\n")
```

```
# Optional: Scatter plot with regression line
ggplot(df_cleaned, aes(x = tem, y = rain)) +
  geom_point(alpha = 0.3, color = "purple") +
  geom_smooth(method = "lm", color = "red") +
  labs(title = paste("Temperature vs Rainfall\nCorrelation =",
    round(correlation, 3)),
    x = "Temperature (°C)",
    y = "Rainfall (mm)") +
  theme_minimal()
```

What this produces:

- Pearson correlation value (e.g., 0.693)
- Interpretation: weak/strong, positive/negative
- Scatter plot showing relationship + regression line

4. Interpretation and Conclusion

The cleaned dataset shows a mean temperature of 25.04 °C and an average monthly rainfall of 201.21 mm. Seasonal patterns align with the South Asian monsoon system, with peak rainfall occurring between June and August and the highest temperatures recorded from May to July. Linear trend analysis reveals a modest long-term warming of approximately 0.56 °C per century and a slight decrease in rainfall of about 19.41 mm per century. However, the low R-squared values indicate that these trends are weak and overshadowed by high natural variability.

This suggests that short-term fluctuations play a significant role in Bangladesh's climate patterns, and long-term trends should be interpreted cautiously. The analysis provides a foundational understanding of historical climatic behavior, but further studies incorporating higher-frequency data, extreme event analysis, additional environmental variables, and regional comparisons would lead to more robust conclusions. Climate insights from this dataset can also support applications such as agricultural planning, disaster management, water resource forecasting, and climate adaptation policy.

5.Full code

```
library(tidyverse)
weather <- read.csv("sorted_temp_and_rain_dataset.csv")

# Just look at what you have
head(weather)

##cleaning
# Load required libraries
library(dplyr)
library(tidyr)

# Read the dataset
df <- read.csv('sorted_temp_and_rain_dataset.csv')
# Display initial info
cat("=== INITIAL DATASET INFO ===\n")
cat("Dataset dimensions:", dim(df), "\n")
cat("Columns:", names(df), "\n")
cat("\nMissing values before cleaning:\n")
print(colSums(is.na(df)))
cat("\nData types:\n")
print(sapply(df, class))

# STEP 1: Remove the 'name' column as requested
cat("\n=== STEP 1: REMOVING 'name' COLUMN ===\n")
df_cleaned <- df %>% select(-name)
cat("'name' column removed successfully\n")

# STEP 2: Handle missing values in numeric columns
cat("\n=== STEP 2: HANDLING MISSING VALUES ===\n")

# Check for missing values in key columns
missing_before <- colSums(is.na(df_cleaned[, c('tem', 'rain', 'Month', 'Year')]))
cat("Missing values before treatment:\n")
print(missing_before)

# Handle missing values in Year column
df_cleaned <- df_cleaned %>%
  mutate(Year = ifelse(is.na(Year),
    approx(seq_along(Year), Year, seq_along(Year))$y,
    Year))

# Handle missing values in Month column (should be 1-12)
df_cleaned <- df_cleaned %>%
  mutate(Month = ifelse(is.na(Month) | Month < 1 | Month > 12,
    NA, Month))

# For temperature (tem) - use linear interpolation
df_cleaned <- df_cleaned %>%
  mutate(tem = ifelse(is.na(tem),
    approx(seq_along(tem), tem, seq_along(tem))$y,
    tem))

# For rainfall (rain) - use linear interpolation
df_cleaned <- df_cleaned %>%
  mutate(rain = ifelse(is.na(rain),
    approx(seq_along(rain), rain, seq_along(rain))$y,
```

```

rain))

# Remove any remaining rows with NA values
df_cleaned <- df_cleaned %>% drop_na()

cat("\nMissing values after cleaning:\n")
print(colSums(is.na(df_cleaned)))

# STEP 3: OUTLIER DETECTION AND REMOVAL
cat("\n=== STEP 3: OUTLIER DETECTION AND REMOVAL ===\n")

# Calculate outlier bounds using IQR method for rainfall
rain_Q1 <- quantile(df_cleaned$rain, 0.25, na.rm = TRUE)
rain_Q3 <- quantile(df_cleaned$rain, 0.75, na.rm = TRUE)
rain_IQR <- rain_Q3 - rain_Q1
lower_bound <- rain_Q1 - 1.5 * rain_IQR
upper_bound <- rain_Q3 + 1.5 * rain_IQR

cat("Rainfall Outlier Detection:\n")
cat("Q1 (25th percentile):", rain_Q1, "\n")
cat("Q3 (75th percentile):", rain_Q3, "\n")
cat("IQR:", rain_IQR, "\n")
cat("Lower bound:", lower_bound, "\n")
cat("Upper bound:", upper_bound, "\n")

# Identify outliers
rain_outliers <- df_cleaned %>%
  filter(rain < lower_bound | rain > upper_bound) %>%
  select(Year, Month, rain)

cat("\nIdentified rainfall outliers:\n")
print(rain_outliers)
cat("Number of rainfall outliers:", nrow(rain_outliers), "\n")

# Remove outliers
df_cleaned <- df_cleaned %>%
  filter(rain >= lower_bound & rain <= upper_bound)

cat("Outliers removed from dataset\n")

# STEP 4: Data validation and quality checks
cat("\n=== STEP 4: DATA VALIDATION ===\n")

# Check for reasonable ranges
cat("Temperature range:", range(df_cleaned$tem, na.rm = TRUE), "\n")
cat("Rainfall range:", range(df_cleaned$rain, na.rm = TRUE), "\n")
cat("Month range:", range(df_cleaned$Month, na.rm = TRUE), "\n")
cat("Year range:", range(df_cleaned$Year, na.rm = TRUE), "\n")

# Check for outliers in temperature (values outside 3 standard deviations)
tem_mean <- mean(df_cleaned$tem, na.rm = TRUE)
tem_sd <- sd(df_cleaned$tem, na.rm = TRUE)
tem_outliers <- sum(abs(df_cleaned$tem - tem_mean) > 3 * tem_sd, na.rm = TRUE)

cat("Potential temperature outliers (>3 SD):", tem_outliers, "\n")

# Check for negative rainfall (impossible values)
negative_rain <- sum(df_cleaned$rain < 0, na.rm = TRUE)
cat("Negative rainfall values:", negative_rain, "\n")

# Fix negative rainfall values
df_cleaned <- df_cleaned %>%

```

```

mutate(rain = ifelse(rain < 0, 0, rain))

# STEP 5: Final dataset info
cat("\n=== FINAL CLEANED DATASET ===\n")
cat("Final dimensions:", dim(df_cleaned), "\n")
cat("Rows removed:", nrow(df) - nrow(df_cleaned), "\n")
cat("\nFirst few rows of cleaned data:\n")
print(head(df_cleaned))

# Summary statistics
cat("\nSummary statistics of cleaned data:\n")
print(summary(df_cleaned))

# Save cleaned dataset
write.csv(df_cleaned, "cleaned_climate_data.csv", row.names = FALSE)
cat("\nCleaned dataset saved as 'cleaned_climate_data.csv'\n")

#####cleaning_end #####

#Visualization:

# Load required libraries
library(ggplot2)
library(dplyr)
library(tidyr)
library(lubridate)

# Read the cleaned dataset (use your cleaned data)
df <- read.csv('cleaned_climate_data.csv')

# Convert to proper date format
df$Date <- as.Date(paste(df$Year, df$Month, "15", sep = "-"))

# =====
# TIME SERIES PLOTS
# =====

# 1. Temperature Time Series
p1 <- ggplot(df, aes(x = Date, y = tem)) +
  geom_line(color = "red", alpha = 0.7) +
  geom_smooth(method = "loess", color = "darkred", se = FALSE) +
  labs(title = "Temperature Time Series (1901-2023)",
       x = "Year",
       y = "Temperature (°C)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

print(p1)

# 2. Rainfall Time Series
p2 <- ggplot(df, aes(x = Date, y = rain)) +
  geom_line(color = "blue", alpha = 0.7) +
  geom_smooth(method = "loess", color = "darkblue", se = FALSE) +
  labs(title = "Rainfall Time Series (1901-2023)",
       x = "Year",
       y = "Rainfall (mm)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

print(p2)

```

3. Combined Time Series (Dual Y-axis)

```
p3 <- ggplot(df) +  
  geom_line(aes(x = Date, y = tem, color = "Temperature"), alpha = 0.7) +  
  geom_line(aes(x = Date, y = rain/20, color = "Rainfall"), alpha = 0.7) +  
  scale_y_continuous(  
    name = "Temperature (°C)",  
    sec.axis = sec_axis(~.*20, name = "Rainfall (mm)")  
  ) +  
  scale_color_manual(values = c("Temperature" = "red", "Rainfall" = "blue")) +  
  labs(title = "Temperature and Rainfall Trends (1901-2023)",  
    x = "Year",  
    color = "Variable") +  
  theme_minimal() +  
  theme(plot.title = element_text(hjust = 0.5),  
    legend.position = "bottom")
```

```
print(p3)
```

```
# =====  
# SEASONAL ANALYSIS  
# =====
```

4. Seasonal Temperature Pattern (Boxplot by Month)

```
p4 <- ggplot(df, aes(x = factor(Month), y = tem)) +  
  geom_boxplot(fill = "lightcoral", alpha = 0.7) +  
  labs(title = "Seasonal Temperature Variation by Month",  
    x = "Month",  
    y = "Temperature (°C)") +  
  scale_x_discrete(labels = month.abb) +  
  theme_minimal() +  
  theme(plot.title = element_text(hjust = 0.5))
```

```
print(p4)
```

5. Seasonal Rainfall Pattern (Boxplot by Month)

```
p5 <- ggplot(df, aes(x = factor(Month), y = rain)) +  
  geom_boxplot(fill = "lightblue", alpha = 0.7) +  
  labs(title = "Seasonal Rainfall Variation by Month",  
    x = "Month",  
    y = "Rainfall (mm)") +  
  scale_x_discrete(labels = month.abb) +  
  theme_minimal() +  
  theme(plot.title = element_text(hjust = 0.5))
```

```
print(p5)
```

6. Average Monthly Temperature and Rainfall

```
monthly_avg <- df %>%  
  group_by(Month) %>%  
  summarise(  
    Avg_Temperature = mean(tem, na.rm = TRUE),  
    Avg_Rainfall = mean(rain, na.rm = TRUE)  
  )
```

```
p6 <- ggplot(monthly_avg) +  
  geom_line(aes(x = Month, y = Avg_Temperature, color = "Temperature"), size = 1) +  
  geom_line(aes(x = Month, y = Avg_Rainfall/10, color = "Rainfall"), size = 1) +  
  scale_x_continuous(breaks = 1:12, labels = month.abb) +  
  scale_y_continuous(  
    name = "Average Temperature (°C)",  
    sec.axis = sec_axis(~.*10, name = "Average Rainfall (mm)")  
  )
```

```
) +
scale_color_manual(values = c("Temperature" = "red", "Rainfall" = "blue")) +
labs(title = "Average Monthly Temperature and Rainfall Patterns",
      x = "Month",
      color = "Variable") +
theme_minimal() +
theme(plot.title = element_text(hjust = 0.5),
      legend.position = "bottom")
```

```
print(p6)
```

```
# 7. Heatmap of Monthly Rainfall Patterns
```

```
p7 <- df %>%
  group_by(Year, Month) %>%
  summarise(Avg_Rainfall = mean(rain, na.rm = TRUE)) %>%
  ggplot(aes(x = factor(Month), y = Year, fill = Avg_Rainfall)) +
  geom_tile() +
  scale_fill_gradient(low = "white", high = "blue",
                      name = "Rainfall (mm)") +
  scale_x_discrete(labels = month.abb) +
  labs(title = "Rainfall Heatmap by Year and Month",
      x = "Month",
      y = "Year") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5),
        axis.text.x = element_text(angle = 45, hjust = 1))
```

```
print(p7)
```

```
# 8. Seasonal Decomposition (for last 20 years for clarity)
```

```
recent_data <- df %>% filter(Year >= 2000)
```

```
p8 <- ggplot(recent_data, aes(x = factor(Month), y = tem, group = Year)) +
  geom_line(aes(color = Year), alpha = 0.6) +
  scale_color_gradient(low = "blue", high = "red") +
  scale_x_discrete(labels = month.abb) +
  labs(title = "Temperature Patterns by Year (2000-2023)",
      x = "Month",
      y = "Temperature (°C)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```

```
print(p8)
```

```
# 9. Monthly Distribution (Violin Plots)
```

```
p9 <- ggplot(df, aes(x = factor(Month), y = rain)) +
  geom_violin(fill = "lightblue", alpha = 0.7) +
  labs(title = "Monthly Rainfall Distribution",
      x = "Month",
      y = "Rainfall (mm)") +
  scale_x_discrete(labels = month.abb) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```

```
print(p9)
```

```
# Save all plots
```

```
plots <- list(p1, p2, p3, p4, p5, p6, p7, p8, p9)
```

```
# Save individual plots
```

```
for (i in 1:length(plots)) {
  ggsave(paste0("climate_plot_", i, ".png"), plots[[i]],
```

```

    width = 10, height = 6, dpi = 300)
}

cat("All visualizations completed and saved as PNG files!\n")

# Additional: Summary statistics by season
df <- df %>%
  mutate(Season = case_when(
    Month %in% c(12, 1, 2) ~ "Winter",
    Month %in% c(3, 4, 5) ~ "Spring",
    Month %in% c(6, 7, 8) ~ "Summer",
    Month %in% c(9, 10, 11) ~ "Autumn"
  ))

seasonal_summary <- df %>%
  group_by(Season) %>%
  summarise(
    Avg_Temperature = mean(tem, na.rm = TRUE),
    Avg_Rainfall = mean(rain, na.rm = TRUE),
    Count = n()
  )

cat("\n=== SEASONAL SUMMARY ===\n")
print(seasonal_summary)


# =====
# TREND ANALYSIS
# =====
# Load libraries
library(dplyr)
library(ggplot2)

# Read data
df <- read.csv('cleaned_climate_data.csv')

cat("=== SIMPLE TREND ANALYSIS ===\n\n")

# Temperature trend
temp_trend <- lm(tem ~ Year, data = df)
cat("Temperature Trend:\n")
cat("Change per year:", round(coef(temp_trend)[2], 4), "°C\n")
cat("Change per century:", round(coef(temp_trend)[2] * 100, 3), "°C\n\n")

# Rainfall trend
rain_trend <- lm(rain ~ Year, data = df)
cat("Rainfall Trend:\n")
cat("Change per year:", round(coef(rain_trend)[2], 4), "mm\n")
cat("Change per century:", round(coef(rain_trend)[2] * 100, 3), "mm\n\n")

# Simple trend plot
ggplot(df, aes(x = Year, y = tem)) +
  geom_smooth(method = "lm", color = "red") +
  labs(title = "Simple Temperature Trend", x = "Year", y = "Temperature (°C)") +
  theme_minimal()

ggplot(df, aes(x = Year, y = rain)) +

```



```

geom_smooth(method = "lm", color = "blue") +
labs(title = "Simple Rainfall Trend", x = "Year", y = "Rainfall (mm)") +
theme_minimal()

# =====
# SEASONAL PATTERNS
# =====

cat("=== SEASONAL PATTERNS ===\n\n")

# Monthly averages
monthly_avg <- df %>%
group_by(Month) %>%
summarise(
  Avg_Temp = round(mean(tem), 2),
  Avg_Rain = round(mean(rain), 2)
)

cat("Average by Month:\n")
print(monthly_avg)

# Seasonal plot
ggplot(monthly_avg, aes(x = Month, y = Avg_Temp)) +
  geom_line(color = "red", size = 1) +
  geom_point(color = "red") +
  scale_x_continuous(breaks = 1:12, labels = month.abb) +
  labs(title = "Average Monthly Temperature", x = "Month", y = "Temperature (°C)") +
  theme_minimal()

ggplot(monthly_avg, aes(x = Month, y = Avg_Rain)) +
  geom_line(color = "blue", size = 1) +
  geom_point(color = "blue") +
  scale_x_continuous(breaks = 1:12, labels = month.abb) +
  labs(title = "Average Monthly Rainfall", x = "Month", y = "Rainfall (mm)") +
  theme_minimal()

# =====
# CORRELATION
# =====

cat("=== CORRELATION ANALYSIS ===\n\n")

# Overall correlation
correlation <- cor(df$tem, df$rain)
cat("Overall Correlation (Temperature vs Rainfall):", round(correlation, 3), "\n\n")

# Simple scatter plot
ggplot(df, aes(x = tem, y = rain)) +
  geom_point(alpha = 0.3, color = "purple") +
  geom_smooth(method = "lm", color = "red") +
  labs(title = paste("Temperature vs Rainfall\nCorrelation =", round(correlation, 3)),
    x = "Temperature (°C)", y = "Rainfall (mm)") +
  theme_minimal()

# =====
# BASIC SUMMARY
# =====

cat("=== BASIC SUMMARY ===\n\n")
cat("Dataset Period:", min(df$Year), "-", max(df$Year), "\n")
cat("Total Records:", nrow(df), "\n")
cat("Average Temperature:", round(mean(df$tem), 2), "°C\n")

```

```
cat("Average Rainfall:", round(mean(df$rain), 2), "mm\n")
cat("Hottest Month:", month.abb[which.max(monthly_avg$Avg_Temp)], "\n")
cat("Wettest Month:", month.abb[which.max(monthly_avg$Avg_Rain)], "\n")
```

6. Appendix: Code and repository

The complete R script and processed data files are part of the project repository. Key files: sorted_temp_and_rain_dataset.csv (original), cleaned_climate_data.csv (cleaned), climate_analysis.R, and climate_plots/ (visualizations).